

# 作业：分枝限界法

班级：7 班

姓名：卢瑶

学号：20211018341

## 课后题 (P239 : 6, 7, 9)

6. 简述分枝限界法的搜索策略。 (课程目标 1)

(失步建字)

分枝限界法的搜索策略是 广度优先遍历 (BFS),

通过限界函数可以快速找到一个 可行解 或 最优解.

类似作业分配

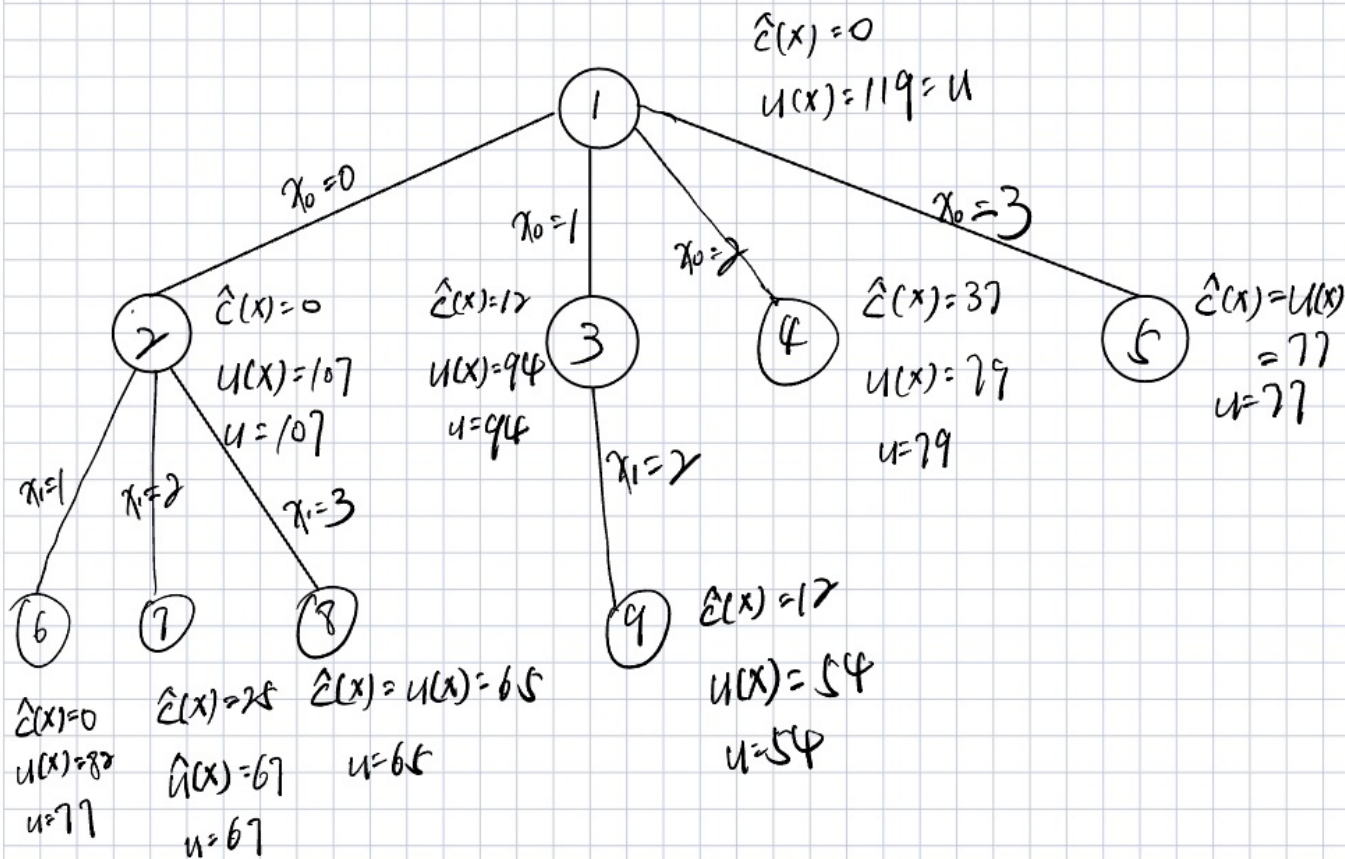
7. 有一个 0/1 背包问题，其中  $n=4$ ，物品重量为 (4, 7, 5, 3)，物品价值为 (40, 42, 25, 12)，背包最大载重量  $W=10$ ，给出采用优先队列式分枝限界法求最优解的过程。（课程目标 1）

限界函数： $\sum_{i=1}^n w_i \leq W$

① 首先对货物按价值排序

| 编号 | w       | v |
|----|---------|---|
| 0  | (3, 12) |   |
| 1  | (5, 25) |   |
| 2  | (4, 40) |   |
| 3  | (7, 42) |   |

② 解空间树如下：



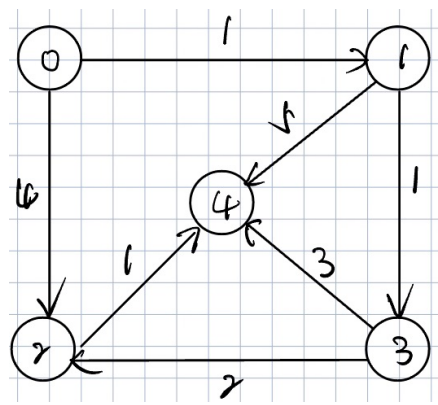
可知： $u=54$  为最小值，分配方案对应最优解。

即取物品 (1, 0, 1, 0)，最大价值为 65。



9. 有一个含  $n$  个顶点（顶点编号为  $0 \sim n-1$ ）的带权图，用邻接矩阵数组  $A$  表示，采用分枝限界法求从起点  $s$  到目标点的最短路径长度，以及具有最短路径长度的路径条数。（课程目标 1）

无向有权图：



```
1  #include <iostream>
2  #include <stdio.h>
3  #include <queue>
4
5  #define INF 0x3f3f3f3f //对应最大值
6
7  using namespace std;
8
9  struct Node{ //学会利用结构体 (class) 进行封装~
10     int vno;
11     int length; //length 表示到当前结点的距离
12
13     bool operator<(const Node &s) const{ //小于号重载, length小的优先
14         return s.length < length;
15     }
16 };
17 //图的邻接矩阵表示 (转化为计算机可以处理的形式)
18 int A[5][5] = {
19     {0, 1, 4, INF, INF},
20     {INF, 0, INF, 1, 5},
21     {INF, INF, 0, INF, 1},
22     {INF, INF, 2, 0, 3},
23     {INF, INF, INF, INF, INF}
24 };
25
26 int cnt = 0; //统计最短路径的个数
27 int length = INF; //记录最大路径的长度
28 int n = 5;
29 //优先队列分枝限界法 (解空间树)
30 void solve(int s, int t){
31     Node e, e1;
32     priority_queue<Node>(q);
33     //结点初始化
34     e.vno = s;
35     e.length = 0;
36     q.push(e); //压队
37     while(!q.empty()){
38         e = q.top(); q.pop(); //优先队列中的第一个结点进行BFS
39         if(e.vno == t){ //递归出口, 到达叶子结点
40             if(e.length < length){
41                 length = e.length; //更新最短路径~(最后一步进行更新)
42                 cnt++;
43             }else if(e.length == length){
44                 cnt++;
45             }else{
46                 for(int i = 0; i < n; i++){ //遍历和e相邻的结点
47                     if(e.length + A[e.vno][i] < length){ //剪枝, 比之前记录的最短路径大的直接跳过即可 (不压入队列)
48                         if(A[e.vno][i] != INF && A[e.vno][i] != 0){
49                             e1.vno = i;
50                             e1.length = e.length + A[e.vno][i];
51                             q.push(e1);
52                         }
53                     }
54                 }
55             }
56         }
57     }
58 }
59
60 int main(int argc, char *argv[]) {
61     int s = 0, t = 4;
62     solve(s, t);
63     if(cnt == 0)
64         printf("结点%d到结点%d没有最短路径\n", s, t);
65     else{
66         printf("结点%d到结点%d存在最短路径\n", s, t);
67         printf("最短路径长度: %d \n最短路径的条数: %d", length, cnt);
68     }
69 }
```

测试效果：

结点0到结点4存在最短路径

最短路径长度：5

最短路径的条数：3